

Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

División de Ciencias de la Ingeniería

Estructura de Datos



# **Catálogo de Productos de Supermercado**

Proyecto Fase 1 – Estructura de Datos

Nombre: Juan Fernando García Natareno

RA: 202230077

**Primer Semestre 2026**

El presente proyecto implementa un sistema de gestión de catálogo de productos para un supermercado, utilizando múltiples estructuras de datos implementados desde cero en C++. El sistema permite realizar operaciones de inserción, eliminación y búsqueda de productos utilizando diferentes estructuras, cada una optimizada para un tipo específico de consulta.

Las estructuras implementadas incluyen listas enlazadas, un árbol AVL para búsqueda binaria por nombre, una tabla hash para acceso directo por código de barra, un árbol B para búsquedas por rango de fechas de caducidad, y un árbol B+ para recorridos eficientes por categoría mediante hojas enlazadas.

Todas las estructuras se mantienen sincronizadas a través de una clase orquestadora(Catalogo) que garantiza la consistencia de los datos mediante un mecanismo de inserción con rollback en caso de un fallo

## Diseño del Sistema

- Arquitectura General:

El sistema sigue una arquitectura en capas: la capa de presentación (menú de consola), la capa lógica (Catálogo como orquestador), y la capa de datos (las 6 estructuras).

| Directorio | Contenido                                                                   |
|------------|-----------------------------------------------------------------------------|
| src/       | Archivos de cabecera (.h) y las implementaciones (.cpp) de todas las clases |
| logs/      | Archivo de errores de carga CSV                                             |
| output/    | Archivos .dot y PNG de visualización                                        |

- Producto (Struct)

Cada producto contiene 7 campos: nombre (string), código de barra (string, clave única), categoría (string), fecha de caducidad (string YYYY-MM-DD), marca (string), precio (double) y stock (int). Los operadores de comparación están sobrecargados: < y > comparan por nombre (para el AVL y lista ordenada), == compara por código de barra (para identificación única)

- Estructuras de datos implementadas

- a. Lista enlazada no ordenada

Implementación clásica de lista enlazada simple con un nodo cabeza. La inserción se realiza al frente en  $O(1)$ . La búsqueda recorre todos los nodos secuencialmente hasta encontrar el producto o llegar al final. Soporta búsqueda tanto por nombre como por código de barras.

- b. Lista enlazada ordenada

Variante de la lista enlazada que mantiene los productos ordenados alfabéticamente por nombre. La inserción recorre la lista para encontrar la posición correcta en  $O(n)$ . La ventaja es el corte temprano en búsquedas: si el nodo actual ya es mayor al buscado, se detiene inmediatamente sin recorrer el resto de la lista,

c. Árbol AVL

Árbol binario de búsqueda auto-balanceado con clave por nombre del producto. Garantiza que la diferencia de altura entre subárboles izquierdo y derecho de cualquier nodo sea como máximo 1 (factor de balance entre -1 y +1).

Cuando una inserción o eliminación viola esta propiedad, el árbol se corrige mediante rotaciones. Se implementan 4 casos: rotación simple izquierda (LL), rotación simple derecha (RR), rotación doble izquierda-derecha (LR) y rotación doble derecha-izquierda (RL). El recorrido in-order produce la lista de productos ordenada alfabéticamente.

d. Tabla Hash

Tabla hash con encadenamiento (chaining) para manejo de colisiones. La clave es el código de barra del producto. Utiliza una función hash polinomial con base 31 y capacidad primo(1009) para una distribución uniforme de las claves. Cada bucket contiene una lista enlazada de nodos. La inserción se realiza al inicio de la cadena. El factor de carga ( $n/\text{capacidad}$ ) se monitorea para evaluar la calidad de la distribución. Con 1050 productos y capacidad 1009, el factor de carga es  $\sim 1.04$  con una cadena máxima de 5 elementos.

e. Árbol B

Árbol B de grado mínimo  $t=3$ , donde cada nodo puede tener entre 2 y 5 claves y entre 3 y 6 hijos. La clave de ordenamiento es la fecha de caducidad en formato ISO (YYYY-MM-DD), lo que permite comparación lexicográfica directa.

Todas las hojas se encuentran al mismo nivel. Cuando un nodo se llena ( $2t-1 = 5$  claves), se divide: la clave del medio sube al padre y las mitades se separan en dos nodos. El árbol crece hacia arriba, no hacia abajo.

La búsqueda por rango es la operación principal: dado un intervalo  $[\text{fechaInicio}, \text{fechaFin}]$ , recorre el árbol encontrando todos los productos cuya fecha de caducidad esté en ese rango. La eliminación implementa las 3 estrategias: uso de predecesor, uso de sucesor, y fusión (merge) de nodos con redistribución entre hermanos.

f. Árbol B+

Árbol B+ de grado mínimo  $t=3$  con clave por categoría del producto. A diferencia del árbol B, los nodos internos solo contienen claves de navegación mientras que todos los datos residen en las hojas. Las hojas están enlazadas entre sí mediante punteros siguiente, formando una lista enlazada.

Esta estructura permite claves duplicadas (múltiples productos de la misma categoría). La búsqueda por categoría navega hasta la hoja correcta en  $O(\log n)$  y luego recorre las hojas enlazadas recolectando todos los productos de esa categoría en  $O(k)$ .

- Análisis de complejidad (Big-O)

| Estructura        | Inserción    | Búsqueda        | Eliminación  |
|-------------------|--------------|-----------------|--------------|
| Lista no ordenada | $O(1)$       | $O(n)$          | $O(n)$       |
| Lista ordenada    | $O(n)$       | $O(n)^*$        | $O(n)$       |
| Árbol AVL         | $O(\log n)$  | $O(\log n)$     | $O(\log n)$  |
| Tabla Hash        | $O(1)$ prom. | $O(1)$ prom.    | $O(1)$ prom. |
| Árbol B           | $O(\log n)$  | $O(\log n + k)$ | $O(\log n)$  |
| Árbol B+          | $O(\log n)$  | $O(\log n + k)$ | $O(\log n)$  |

#### Justificación de las complejidades

**AVL  $O(\log n)$ :** Al mantener el factor de balance entre -1 y +1, la altura del árbol siempre es  $O(\log n)$ . Cada operación recorre un camino desde la raíz hasta una hoja, más las rotaciones que son  $O(1)$  cada una.

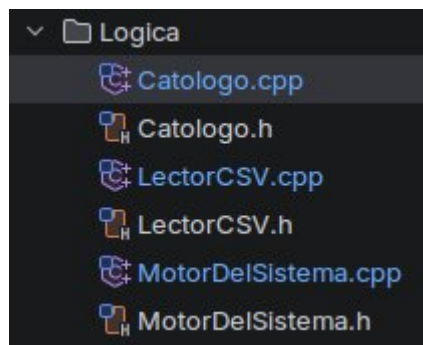
**Hash  $O(1)$  promedio:** La función hash distribuye las claves uniformes. Con factor de carga  $\sim 1.0$  y cadena promedio de 1.6 elementos, la búsqueda toma tiempo constante en promedio. El peor caso  $O(n)$  ocurre solo si todas las claves colisionan en el mismo bucket.

**Árbol B  $O(\log n)$ :** Con grado  $t = 3$ , cada nodo interno tiene entre 3 y 6 hijos. La altura es  $O(\log_t n)$ , que para  $n = 1050$  es aproximadamente 4 niveles. La búsqueda por rango agrega  $O(k)$  por los resultados encontrados.

**Árbol B+  $(\log n + k)$ :** Similar al B, pero las hojas enlazadas permiten recorrer resultados consecutivos sin subir y bajar por el árbol, haciendo el recorrido por categoría más eficiente.

- Lógica del Sistema

La lógica del negocio esta en la parte o en la carpeta “Logica” la cual cuenta con los siguiente archivos.



- o Catologo: Es el núcleo del sistema. Contiene internamente las 6 estructuras de datos y contiene los métodos para las operaciones del sistema
- o LectorCSV: La clase que se encarga de las lecturas de los archivos .csv, implementa métodos para el manejo de comillas y separadores, también maneja los retornos de carro y hace las validaciones de los 7 campos obligatorios para la inserción de los productos y también hace un recuento de errores y en que línea fue hecha y los escribe en un archivo de errores
- o MotorDelSistema: Clase donde se unen todos los métodos del Catologo para que toda la lógica del negocio funciones, también ahí se crea el menú y otras funcionalidades, el método mas importante es el motorDelSistema() el cual donde vive toda la lógica y el que se manda a llamar en el main, para no sobrecargar al mismo

- Visualizacion

El sistema genera archivos .dot para los 3 árboles (AVL, B y B+) que pueden ser renderizados a imágenes PNG usando Graphviz. Cada nodo del archivo .dot incluye información relevante:

- AVL: nombre del producto, altura del nodo y factor de balance.
- B-Tree: fechas de caducidad ordenadas dentro de cada nodo.
- B+ Tree: categorías en nodos internos (navegación) y categoría + nombre en hojas (datos).

Las hojas se distinguen visualmente con diferente color.

Los archivos se generan ejecutando la opción 9 del menú. El programa también invoca automáticamente el comando dot de Graphviz para producir las imágenes PNG.

# TADs

## TAD producto

Representa un producto del supermercado. Es la unidad básica de la información que se almacena en todas las estructuras de datos del sistema. El código de barras es el identificador único de cada producto.

### Atributos privados:

- **String nombre:** Nombre del producto. Se usa como clave en el Arbol AVL y la Lista Ordenada
- **String codigoBarras:** Código único del producto. Se usa como clave en la Tabla Hash y como identificador para eliminaciones
- **String categoria:** Categoría del producto. Se utiliza como clave en el Árbol B+
- **String fechaExpiracion:** Fecha de caducidad en el formato YYYY-MM-DD. Se utiliza como clave para el Árbol B para búsquedas por rango.
- **String marca:** Marca del producto.
- **Double precio:** Precio del producto.
- **Int stock:** Cantidad disponible en el inventario.

### Métodos públicos:

- **Productos():** Constructor por defecto. Inicializa precio en 0.0 y stock en 0.
- **Producto(nombre, codigoBarras, categoria, fechaExpiracion, marca, precio, stock):** Constructor con parámetros. Inicializa todos los campos del producto.
- **mostrar():** Imprime en consola toda la información del producto en formato legible con bordes.
- **operator<(otro):** Compara dos productos por nombre en orden alfabético. Retorna true si este producto es menor.
- **operator>(otro):** Compara dos productos por nombre en orden alfabético inverso.
- **operator==(otro):** Compara dos productos por código de barras. Retorna true si tienen el mismo código

## TAD nodo

Nodo generico para las listas enlazadas simples (ordenada y no ordenada). Contiene un Producto y un puntero al siguiente nodo en la cadena.

### Atributos privados

- **Producto data:** El producto almacenado en este nodo.
- **Nodo\* siguiente:** Puntero al siguiente nodo de la lista. Es nullptr si es el ultimo nodo.

### Métodos públicos

- **Nodo(producto):** Constructor que recibe un Producto, lo almacena en data e inicializa siguiente como nullptr.
- **~Nodo():** Destructor. Libera la memoria del nodo.

## TAD AVLNode

Nodo especializado para el arbol AVL. Ademas del Producto y los punteros a hijos izquierdo y derecho, almacena la altura del subarbol para calcular el factor de balance necesario para las rotaciones.

### Atributos privados

- **Producto data:** El producto almacenado en este nodo del arbol.
- **AVLNode\* izquierdo:** Puntero al hijo izquierdo (productos con nombre menor).
- **AVLNode\* derecho:** Puntero al hijo derecho (productos con nombre mayor).
- **int altura:** Altura del subarbol enraizado en este nodo. Una hoja tiene altura 1.

### Métodos públicos

- **AVLNode(producto):** Constructor que almacena el producto, inicializa izquierdo y derecho como nullptr, y altura como 1.
- **~AVLNode():** Destructor del nodo.

## TAD HashNodo

Nodo para la tabla hash con encadenamiento. Cada bucket de la tabla contiene una lista enlazada de HashNodos para manejar colisiones.

### Atributo privados

- **Producto data:** El producto almacenado en este nodo del bucket.
- **HashNodo\* siguiente:** Puntero al siguiente nodo en la cadena del mismo bucket. Es nullptr si es el ultimo de la cadena.

### Métodos públicos

- **HashNodo(producto):** Constructor que almacena el producto e inicializa siguiente como nullptr.
- **~HashNodo():** Destructor del nodo.

## TAD BNodo

Nodo del árbol B. Puede contener múltiples claves (productos) y múltiples hijos. La cantidad de claves varía entre  $t-1$  y  $2t-1$ , donde  $t$  es el grado mínimo.

### Atributos privados:

- **Producto\* claves:** Arreglo de productos ordenados por fecha de caducidad. Tamaño máximo:  $2t-1$ .
- **BNodo\*\* hijos:** Arreglo de punteros a nodos hijos. Tamaño máximo:  $2t$ . Es nullptr en
- **hojas.**
- **int numClaves:** Cantidad actual de claves almacenadas en este nodo.
- **int grado:** Grado mínimo del árbol ( $t$ ). Define los límites de claves e hijos.
- **bool esHoja:** Indica si el nodo es una hoja (no tiene hijos).

### Metodos publicos:

- **BNodo(grado, esHoja):** Constructor que asigna memoria para los arreglos de claves ( $2t-1$ ) e hijos ( $2t$ ), inicializa numClaves en 0.
- **~BNodo():** Destructor que libera los arreglos de claves e hijos.

## TAD BMasNodo

Nodo del arbol B+. Los nodos internos solo tienen claves de navegación (strings), mientras que las hojas almacenan los datos completos (Productos) y están enlazadas entre sí para recorridos secuenciales.

Atributos privados:

- `string* claves`: Arreglo de categorías como claves de navegación u ordenamiento. Tamaño máximo:  $2t-1$ .
- `BMasNodo** hijos`: Arreglo de punteros a nodos hijos. Solo existe en nodos internos (`nullptr` en hojas).
- `Producto* data`: Arreglo de productos. Solo existe en hojas (`nullptr` en nodos internos).
- `BMasNodo* siguiente`: Puntero al siguiente nodo hoja, formando una lista enlazada. Solo existe en hojas.
- `int numClaves`: Cantidad actual de claves en este nodo.
- `int grado`: Grado mínimo del árbol ( $t$ ).
- `bool esHoja`: Indica si es una hoja (contiene datos) o nodo interno (solo navegación).

Métodos públicos

- `BMasNodo(grado, esHoja)`: Constructor que asigna memoria según el tipo: si es hoja crea arreglos de claves y data; si es interno, crea arreglos de claves e hijos.
- `~BMasNodo()`: Destructor que libera los arreglos correspondientes.

## TAD ListaEnlazada – Lista No Ordenada

Lista enlazada simple no ordenada. Los productos se insertan al frente en  $O(1)$ . Sirve como estructura base comparativa para medir el costo de la búsqueda secuencial frente a las estructuras más eficientes como el AVL o la tabla hash.

Atributos privados:

- `Nodo* cabeza`: Puntero al primer nodo de la lista. Es `nullptr` si la lista está vacía.
- `int tamaño`: Contador del número total de productos en la lista.

Métodos públicos

- `insertar(Producto)`: Crea un nuevo nodo con el producto y lo coloca al frente de la lista, actualizando cabeza. Complejidad  $O(1)$ .

- `buscarPorNombre(string)`: Recorre la lista desde la cabeza comparando nombres. Retorna un puntero al producto o `nullptr` si no existe. Complejidad  $O(n)$ .
- `buscarPorBarcode(string)`: Recorre la lista comparando codigos de barra. Retorna puntero al producto o `nullptr`. Complejidad  $O(n)$ .
- `remove(string barcode)`: Busca el nodo con ese codigo de barras, lo desenlaza de la cadena y libera su memoria. Complejidad  $O(n)$ .
- `mostrar()`: Recorre toda la lista e imprime cada producto con su numero de posicion.
- `getTamano()`: Retorna el valor de tamano.
- `estaVacia()`: Retorna `true` si cabeza es `nullptr`.
- `getCabeza()`: Retorna el puntero a la cabeza de la lista.

## TAD ArbolAVL

Arbol binario de busqueda auto-balanceado con clave por nombre del producto. Garantiza que el factor de balance (altura del subarbol izquierdo menos altura del derecho) de cada nodo sea -1, 0 o +1. Cuando una insercion o eliminacion viola esta propiedad, el arbol se corrige mediante rotaciones simples o dobles.

Atributos privados:

- `AVLNodo* raiz`: Puntero a la raiz del arbol. Es `nullptr` si el arbol esta vacio.
- `int tamano`: Cantidad total de productos almacenados en el arbol.

## Métodos públicos

- `insertar(Producto)`: Inserta el producto en la posicion correcta segun su nombre (como BST), luego sube por el camino actualizando alturas y aplicando rotaciones si algun nodo queda desbalanceado. Rechaza nombres duplicados. Complejidad  $O(\log n)$ .
- `buscar(string nombre)`: Busqueda binaria: en cada nodo decide si ir a la izquierda (nombre menor) o derecha (nombre mayor). Complejidad  $O(\log n)$ .
- `remove(string nombre)`: Elimina el nodo usando las 3 reglas de BST (sin hijos, un hijo, dos hijos con sucesor in-order), luego rebalancea subiendo por el camino. Complejidad  $O(\log n)$ .
- `enOrden()`: Recorrido in-order (izquierda, raiz, derecha) que produce los productos en orden alfabetico. Complejidad  $O(n)$ .
- `generarDot(string archivo)`: Genera un archivo `.dot` para Graphviz mostrando cada nodo con su nombre, altura y factor de balance.

## TAD TablaHash

Tabla hash que usa encadenamiento (chaining) para resolver colisiones. La clave es el código de barra del producto. Cada posición del arreglo (bucket) contiene una lista enlazada de HashNodos. Utiliza una función hash polinomial con base 31 y capacidad primo (1009) para buena distribución.

Atributos privados:

- HashNodo\*\* tabla: Arreglo de punteros a cadenas de HashNodos. Cada posición es un bucket que puede contener múltiples productos que colisionaron.
- int capacidad: Tamaño del arreglo (número primo, por defecto 1009). Un primo reduce patrones de colisión.
- int tamaño: Cantidad total de productos insertados en la tabla.

Métodos públicos

- **insertar(Producto):** Calcula el índice con la función hash, verifica que no exista el barcode en esa cadena, e inserta al inicio del bucket. Complejidad  $O(1)$  promedio.
- **buscar(string barcode):** Calcula el hash del barcode, va directo al bucket y recorre la cadena (que en promedio es corta). Complejidad  $O(1)$  promedio.
- **remove(string barcode):** Localiza el bucket, recorre la cadena y desenlaza el nodo. Complejidad  $O(1)$  promedio.
- **mostrarEstadisticas():** Muestra capacidad, cantidad de elementos, factor de carga, buckets usados, cadena más larga y cadena promedio.
- **mostrar():** Recorre todos los buckets e imprime cada producto encontrado.
- **getFactorCarga():** Retorna tamaño/capacidad. Valores menores a 0.7 son ideales; mayores a 1.0 indican muchas colisiones.
- **funcionHash(string clave):** Función hash polinomial: recorre cada carácter multiplicando por 31 y sumando. Aplica módulo por la capacidad para obtener un índice válido.

## TAD ArbolB

Arbol B de grado minimo  $t=3$ , donde cada nodo puede contener entre 2 y 5 claves y entre 3 y 6 hijos. La clave de ordenamiento es la fecha de caducidad en formato ISO (YYYY-MM-DD), lo que permite comparacion lexicografica directa. Todas las hojas estan al mismo nivel. Cuando un nodo se llena, se divide y la clave del medio sube al padre.

### Atributos privados

- **BNodo\* raiz:** Puntero a la raiz del arbol. Es nullptr si esta vacio.
- **int grado:** Grado minimo ( $t=3$ ). Define que cada nodo tiene entre  $t-1$  y  $2t-1$  claves.
- **int tamano:** Cantidad total de productos en el arbol.

### Métodos públicos

- **insertar(Producto):** Si la raiz esta llena, crea una nueva raiz y divide la antigua. Luego baja al hijo correcto e inserta. Permite fechas duplicadas. Complejidad  $O(\log n)$ .
- **buscar(string fecha):** En cada nodo busca la primera clave mayor o igual a la fecha. Si la encuentra, retorna el producto. Si no, baja al hijo correspondiente. Complejidad  $O(\log n)$ .
- **busquedaRango(string inicio, string fin):** Recorre el arbol mostrando todos los productos cuya fecha este en el intervalo  $[\text{inicio}, \text{fin}]$ . Complejidad  $O(\log n + k)$  donde  $k$  son los resultados.
- **remove(string fecha):** Elimina usando tres estrategias: si la clave esta en una hoja, la quita directamente. Si esta en un nodo interno, la reemplaza por su predecesor o sucesor. Si el hijo destino tiene pocas claves, primero lo llena tomando de hermanos o fusionando. Complejidad  $O(\log n)$ .
- **generarDot(string archivo):** Genera archivo .dot mostrando cada nodo con sus fechas ordenadas.
- **separarHijo(padre, i, hijo):** Divide un hijo lleno: la clave del medio sube al padre, la mitad izquierda queda en el hijo original y la mitad derecha en un nuevo nodo.
- **insertarNoCompleto(nodo, producto):** Inserta en un nodo que no esta lleno. Si es hoja, desplaza claves. Si es interno, baja al hijo correcto (dividiendo primero si esta lleno).
- **llenar(nodo, idx):** Asegura que el hijo en posicion  $\text{idx}$  tenga al menos  $t$  claves, tomando prestado de hermanos o fusionando.
- **fusionar(nodo, idx):** Fusiona  $\text{hijos}[\text{idx}]$  con  $\text{hijos}[\text{idx}+1]$ , bajando la clave separadora del padre.

## TAD ArbolB+

Arbol B+ de grado minimo  $t=3$  con clave por categoria. Los nodos internos solo contienen claves de navegacion (strings de categoria), mientras que todos los datos (Productos completos) residen exclusivamente en las hojas. Las hojas estan enlazadas entre si mediante un puntero next, formando una lista que permite recorridos secuenciales eficientes. Permite claves duplicadas (múltiples productos de la misma categoria).

Atributos privados:

- **BMasNodo\* raiz:** Puntero a la raiz del arbol. Es nullptr si esta vacio.
- **int grado:** Grado minimo ( $t=3$ ).
- **int tamano:** Cantidad total de productos almacenados.

## Métodos públicos

- **insertar(Producto):** Navega hasta la hoja correcta segun la categoria del producto e inserta ahi. Si la hoja se llena, se divide: la primera clave del nuevo nodo se copia al padre (a diferencia del B donde sube). Las hojas se enlazan automaticamente. Complejidad  $O(\log n)$ .
- **buscarPorCategoria(string):** Navega por los nodos internos hasta llegar a la hoja que contiene la categoria buscada, luego recorre las hojas enlazadas recolectando todos los productos de esa categoria. Complejidad  $O(\log n + k)$ .
- **remove(string categoria, string barcode):** Elimina un producto especifico identificado por categoria y barcode. Necesita ambos porque la categoria no es unica. Complejidad  $O(\log n)$ .
- **mostrarTodo():** Navega hasta la primera hoja (la mas a la izquierda) y recorre todas las hojas usando los punteros next, imprimiendo cada producto. Demuestra la ventaja del enlace entre hojas. Complejidad  $O(n)$ .
- **generarDot(string archivo):** Genera archivo .dot donde los nodos internos se muestran en un color y las hojas en otro, con las categorias y nombres de productos.
- **separarHijo(padre, i, hijo):** Divide un hijo lleno. Si es hoja: copia la primera clave del nuevo nodo al padre y enlaza las hojas. Si es interno: sube la clave del medio (como en B normal).
- **insertarNoLleno(nodo, producto):** Inserta en un nodo no lleno. Si es hoja, desplaza datos. Si es interno, baja al hijo correcto.
- **fusionar(nodo, idx):** Fusiona dos nodos hijos y actualiza el enlace de hojas si aplica.

## TAD Catalogo

Clase central que contiene las 6 estructuras de datos y garantiza que todas almacenen exactamente los mismos productos en todo momento. Implementa insercion atomica con rollback: si una insercion falla en la estructura N, se deshacen las inserciones exitosas en las estructuras 1 a N-1, evitando inconsistencias

### Atributos privados

- **ListaEnlazada listaProductos:** Lista no ordenada para busqueda secuencial base.
- **ListaOrdenada listaOrdenada:** Lista ordenada por nombre con corte temprano.
- **ArbolAVL arbolAVL:** Arbol AVL para busqueda binaria por nombre y listado ordenado.
- **TablaHash tablaHash:** Tabla hash para acceso  $O(1)$  por codigo de barras.
- **ArbolB arbolB:** Arbol B para busquedas por rango de fechas de caducidad.
- **ArbolBMas arbolBMas:** Arbol B+ para busquedas por categoria con hojas enlazadas.
- **int totalProductos:** Contador global de productos en el sistema.

### Métodos públicos

- **agregarProducto(Producto):** Inserta el producto en las 6 estructuras en orden (lista, lista ordenada, AVL, hash, B, B+). Si falla en la estructura N, elimina de las N-1 anteriores (rollback). Complejidad  $O(\log n)$ .
- **removerProducto(string barcode):** Busca el producto por barcode en la tabla hash ( $O(1)$ ), guarda una copia de sus datos, y lo elimina de cada estructura usando la clave apropiada. Complejidad  $O(\log n)$ .
- **buscarPorNombre(string):** Delega la busqueda al ArbolAVL. Complejidad  $O(\log n)$ .
- **buscarPorBarcode(string):** Delega la busqueda a la TablaHash. Complejidad  $O(1)$  promedio.
- **buscarPorCategoria(string):** Delega al ArbolBMas. Complejidad  $O(\log n + k)$ .
- **buscarPorRangoFechas(string inicio, string fin):** Delega al ArbolB. Complejidad  $O(\log n + k)$ .
- **listarPorNombre():** Ejecuta el recorrido in-order del ArbolAVL para mostrar productos en orden alfabetico. Complejidad  $O(n)$ .
- **generarArchivosDot():** Genera los 3 archivos .dot (AVL, B, B+) y ejecuta Graphviz para crear las imagenes PNG.
- **getTotal():** Retorna totalProductos.
- **estaVacio():** Retorna true si totalProductos es 0.

## TAD LectorCSV

Clase encargada de leer archivos CSV, parsear cada línea extrayendo los 7 campos del producto, validar que los datos sean correctos, y cargar los productos válidos al Catálogo. Los errores se registran en un archivo de log para revisión posterior.

### Métodos públicos

- **cargar(string archivo, Catalogo& catalogo):** Abre el archivo CSV, salta el header, y para cada línea: limpia retornos de carro, parsea los 7 campos, valida y llama a catalogo.agregarProducto(). Al final imprime resumen de cargados, duplicados y errores. Retorna la cantidad de productos cargados exitosamente.
- **parsearLinea(string linea, Producto&):** Extrae los 7 campos de una línea CSV respetando campos entre comillas. Convierte precio a double y stock a int. Retorna false si el formato es inválido.
- **validar(Producto, int numLinea, string& error):** Verifica que nombre, barcode y categoría no estén vacíos, que la fecha tenga 10 caracteres, y que precio y stock no sean negativos. Retorna false y llena el mensaje de error si falla alguna validación.
- **quitarComillas(string campo):** Elimina espacios al inicio y final del campo, y quita las comillas si están presentes.

## TAD MotorDelSistema

Clase que encapsula toda la lógica de la interfaz de consola del sistema. Contiene un Catálogo internamente y expone un único método público motorDelSistema() que ejecuta el ciclo principal del menú interactivo. En el main() solo se instancia esta clase y se llama a ese método

### Métodos públicos

- **motorDelSistema():** Método público principal. Muestra el encabezado, entra en un ciclo que muestra el menú, lee la opción del usuario y ejecuta la acción correspondiente hasta que el usuario elija salir.
- **mostrarEncabezado():** Imprime el banner del sistema con nombre del proyecto y universidad.
- **mostrarMenu():** Imprime las 10 opciones del menú principal con formato de tabla.
- **cargarCSV():** Solicita la ruta del archivo CSV al usuario, crea un LectorCSV y llama a cargar().

- **agregarProducto():** Solicita los 7 campos del producto al usuario por consola y llama a `catalogo.agregarProducto()`.
- **eliminarProducto():** Solicita el codigo de barras y llama a `catalogo.removerProducto()`.
- **buscarPorNombre():** Solicita el nombre y muestra el resultado de `catalogo.buscarPorNombre()`.
- **buscarPorBarcode():** Solicita el barcode y muestra el resultado de `catalogo.buscarPorBarcode()`.
- **buscarPorCategoria():** Solicita la categoria y llama a `catalogo.buscarPorCategoria()`.
- **buscarPorRangoFechas():** Solicita fecha inicio y fin, llama a `catalogo.buscarPorRangoFechas()`.
- **generarDot():** Llama a `catalogo.generarArchivosDot()` y ejecuta los comandos de Graphviz para generar las imagenes PNG.
- **mostrarEstadisticasHash():** Llama a `catalogo.getTablaHash().mostrarEstadisticas()`.

## Diagrama de Clases

